SCE 672: Advanced Robotics

Term Project Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

Student: Ameen Ur Rehman (202527430) **Marks:** 10
Instructor: Prof. Aamer Iqbal Bhatti
Institution: King Fahd University of Petroleum & Minerals
Code and data: `https://github.com/AmeenUrRehman/RoboMaster-Advanced-Robotics` (private repository; folder `Assignment 2`)

Abstract

A RoboMaster EP searches for a floor target (Tag 3), aligns with it, follows a smooth trajectory generated with differential flatness, stops 20 cm in front of it, and logs the four wheel-motor voltages once per second. The solution is a ROS 2 package (`rm_flatness_nav`) built from small, tested pieces: a closed-form flatness inversion and a cubic boundary-value solver, a four-state supervisory state machine, an AprilTag detector with pose recovery, a wheel-rate and armature-voltage model, and a ROS 2 node that connects them to the robot. All hardware results come from real logs on the robot; every terminal figure is rendered from the captured text of the real command, and every plot comes from the real CSV files. Two complete autonomous runs over USB ended with the state `ARRIVED`, all four voltages exactly 0.000 V at the halt, a peak speed of 0.137 m/s (limit 0.16 m/s) and a measured stop gap of 20 cm in both runs. The report also documents what did *not* work (two earlier runs that never halted), what was learned from the camera (it is fixed to the chassis and does not tilt with the arm), and where the setup deviates from the assignment text (an upright tag, a tag about 1.3 m away, and a stop that comes from the planned trajectory rather than from a camera reading).

Summary of results

Quantity	Result
Target	Tag 3, `tag36h11` ID 3, black square 0.15 m, upright
Trajectory horizon (Eq. 3), run 4	$T = 1.6\,(1.28-0.20)/0.16 = 10.8$ s planned, 10.9 s measured
Peak forward speed (both runs)	0.137 m/s (limit 0.16 m/s)
Final motor voltages	$V_1 = V_2 = V_3 = V_4 = 0.000$ V at the halt
Measured stop gap (tape measure)	20 cm in run 3 and in run 4
Search / align / chase duration, run 4	11.4 s / 2.5 s / 10.9 s (total 25.2 s)
Unit tests	18 of 18 pass
Camera tilt (measured from images)	about 20° downward, fixed to the chassis

Mission and system overview

The mission

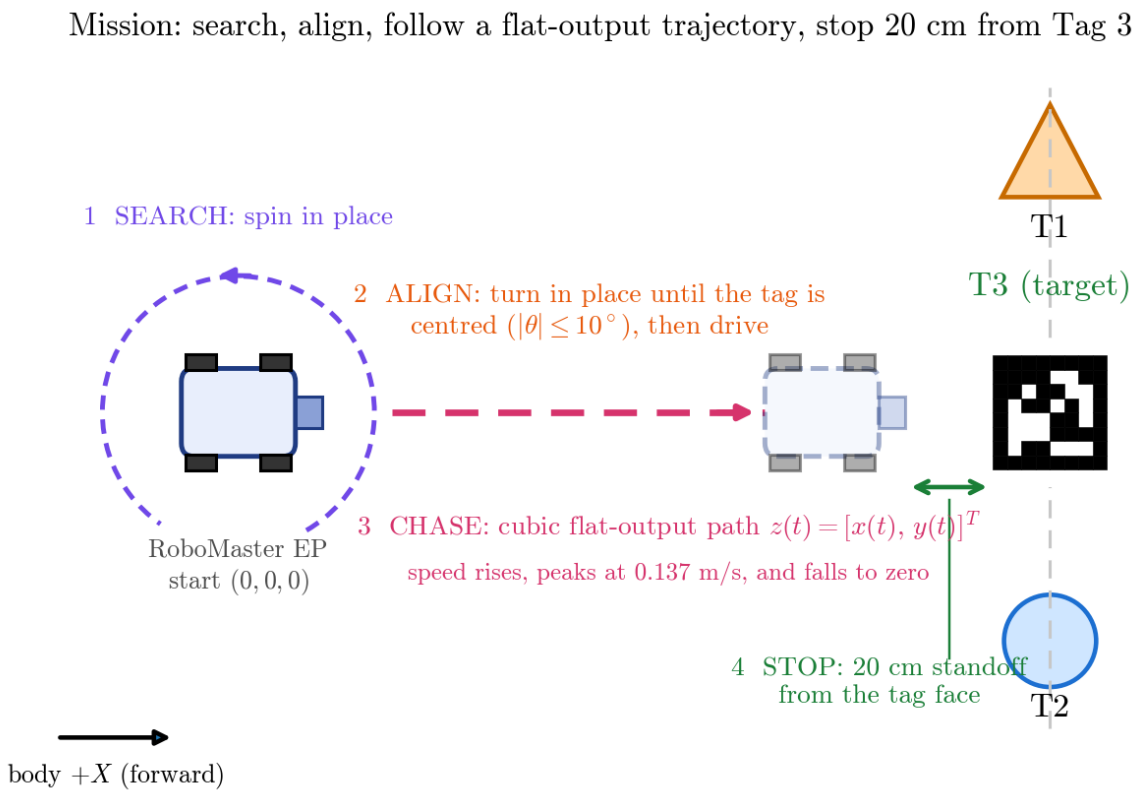


Figure 1: The four behaviours. The tags are drawn as simple shapes for clarity (Tag 1 triangle, Tag 2 circle, Tag 3 the square AprilTag); in the experiments only Tag 3, a real `tag36h11` AprilTag, was placed in front of the robot. The flat-output path of phase 3 is a straight line in the robot frame because the robot first turns until the tag is centred.

The tags

The assignment names three targets, T_1, T_2 and T_3, without specifying a marker family or size. The detector therefore uses the standard AprilTag family `tag36h11`, which OpenCV provides, with the identifier as a parameter (`tag_id`, default 3). Printable A4 sheets for the three tags (IDs 1, 2, 3, black square exactly 150 mm) were generated with `Assets/make_assets.py`; each PDF was rendered back and decoded with OpenCV to check the identifier and the 150 mm size (measured 150.1 mm). The experiments used Tag 3 only.

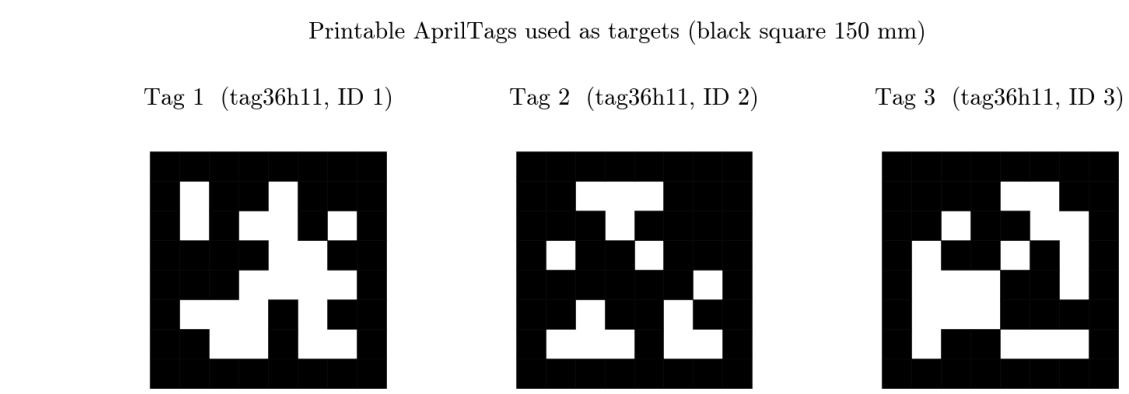


Figure 2: Printable AprilTags (`tag36h11`, IDs 1, 2, 3).

Software architecture

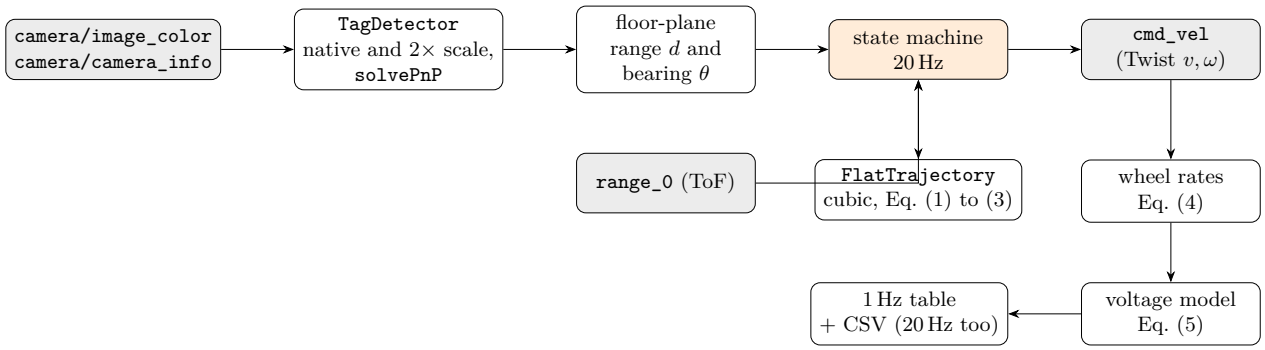


Figure 3: Data flow of `flatness_nav`. Grey boxes are ROS interfaces of the `robomaster_ros` driver.

File	Role
`flatness.py`	flatness inversion, `solve_flat_polynomial` (`numpy.linalg.solve`), time horizon, `FlatTrajectory`
`wheel_voltage.py`	wheel rates (Eq. 4) and the armature voltage model (Eq. 5)
`state_machine.py`	SEARCHING, ALIGNING, CHASING, ARRIVED (pure logic, unit tested)
`tag_detector.py`	AprilTag detection, pose with `solvePnP`, floor-plane range and bearing
`nav_node.py`	the ROS 2 node: camera, ToF, `cmd_vel`, logging
`camera_pose.py`, `sim.py`	visibility predictor; offline simulation (not hardware data)

The control loop runs at 20 Hz. The code is in the repository (see the link at the top) and the main files are listed in Appendix A.

Data provenance and run history

Four real runs of the full system exist. They are listed because the two failures led directly to design changes. The columns are computed from the logs by `scripts/make_tables.py`.

Run	logged	link	final state	search (s)	first seen (m)	chase d_0 (m)	chase (s)	peak v (m/s)	last seen (m)
1	16:00	Wi-Fi AP	SEARCHING (no halt)	2.3	0.906	0.906	–	0.137	0.535
2	16:04	Wi-Fi AP	SEARCHING (no halt)	44.9	1.189	1.061	–	0.137	0.605
3	18:36	USB	ARRIVED	35.9	1.375	1.367	11.8	0.137	0.379
4	19:14	USB	ARRIVED	11.4	1.280	1.280	10.9	0.137	0.351

Run history. The four real runs. "First seen" and "last seen" are the camera distances at the first and last detection that appear in the 1 Hz log.

- **Runs 1 and 2 (before the fix).** The robot detected the tag, aligned and started the chase, but when the tag left the image it returned to SEARCHING and never halted. In those runs the helper output of the session shows a configured tag size of 0.20 m while the tag is 0.15 m, so the distances reported there are about $0.20/0.15 = 1.33$ times too large.
- **The fix.** If the tag disappears *during* a chase and was last seen within 1.8 m, the planned trajectory simply continues from its time base (it already ends at the standoff).
- **Runs 3 and 4 (after the fix, over USB, tag size 0.15 m).** Both ended in `ARRIVED`. They are the runs analysed below; run 4 is the one shown in Table 1 and the plots.

Problem 1: Visual search, in-place centering and standoff guard

(i) Downward camera setup

Why the camera must look down. The tags lie on the floor, so the useful part of the field of view is the floor. A level camera at height h sees a floor point at distance d at a depression angle $\arctan(h/d)$. With the vertical half field of view of the stream (about 30°, from $f_y \approx 316$ px at 360 p), a level camera loses the floor in front of it once $\arctan(h/d) > 30°$, i.e. closer than $h/\tan 30°$, so the tag would leave the image before the 20 cm standoff. Tilting the camera down by β moves the visible floor band closer, to $h/\tan(30° + \beta)$, and makes the view of the floor less grazing, which reduces the foreshortening of a flat tag.

What was measured. The driver was commanded to the pose $(X, Y) = (105, 30)$ mm (the driver names the vertical axis z) and the arm read back $x = 0.105$ m, $z = 0.03$ m. Camera frames were then captured at this pose and at other arm positions (Figure 4): the frames *did not change* when only the arm moved. The robot description in TF attaches the camera to the arm, but on this robot the camera is fixed to the chassis, so its tilt is not set by the arm pose, and that part of TF is not used. In the images the horizon (the bases of the far walls) lies roughly 100 to 130 px above the image centre, which corresponds to a downward tilt of about $20° \pm 3°$ (the tilt is read from the images and is an estimate). The gripper claw is visible at the bottom centre of every frame, so a floor tag directly ahead and closer than roughly 0.3 m is hidden by it. The arm position $(105, 30)$ mm is a low, retracted pose; the ToF beam clearance was checked from the run log: the sensor module was enabled and in the whole run it never returned a reading below its maximum range, so neither the arm nor the gripper was in its beam. The camera height above the floor was not measured.

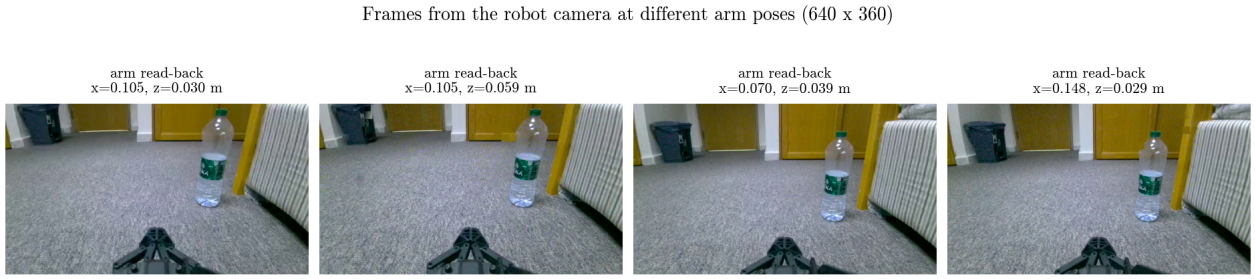


Figure 4: Real frames at four arm poses (the arm read-back is printed above each frame). Within each pair of frames (left two, right two) the arm moved and the image did not change. The two pairs differ slightly from each other (the horizon is higher in the left pair), possibly because the robot was moved between the captures.

The detector on a real frame. Figure 5 shows the project detector decoding the real tag from a frame of this camera. At that frame the tag is 61 px wide and the pose estimate gives a range of 0.81 m.

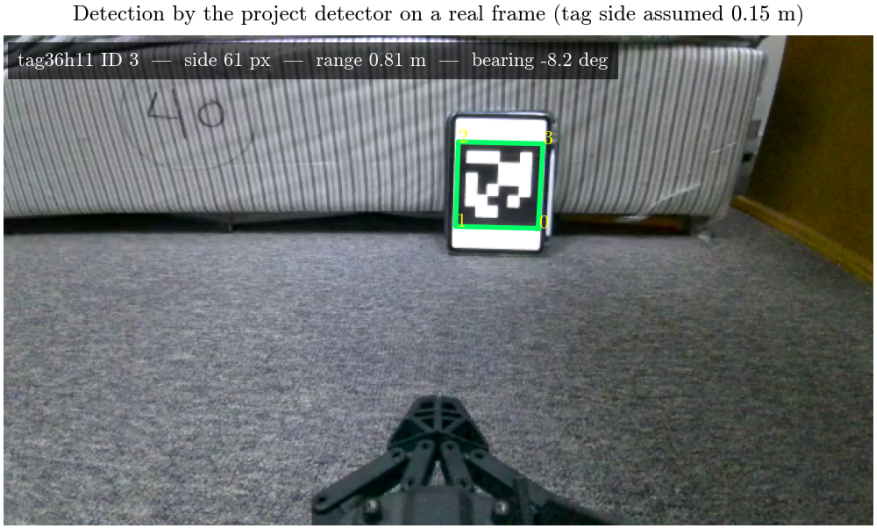


Figure 5: Detection of `tag36h11` ID 3 on a real frame. The distance assumes a 0.15 m black square and was not checked with a tape.

Why the tag was upright, and how far it is visible. A flat floor tag seen from a camera about 0.25 m above the floor is strongly foreshortened: for a 15 cm tag the model of Figure 6 (camera height assumed) gives only 11 px of height at 1 m and 5 px at 1.5 m, which a detector cannot read. An upright tag keeps its full width: $317 \cdot 0.15/d$ pixels, i.e. 47 px at 1 m. The detection limit was measured by shrinking a real frame: the tag was found at 41 px at native size and not at 33 px; after enlarging the image twice before detection it was found at 33 px and 27 px and not at 24 px. The detector therefore tries native size first and the doubled image if nothing is found (21 ms per empty frame).

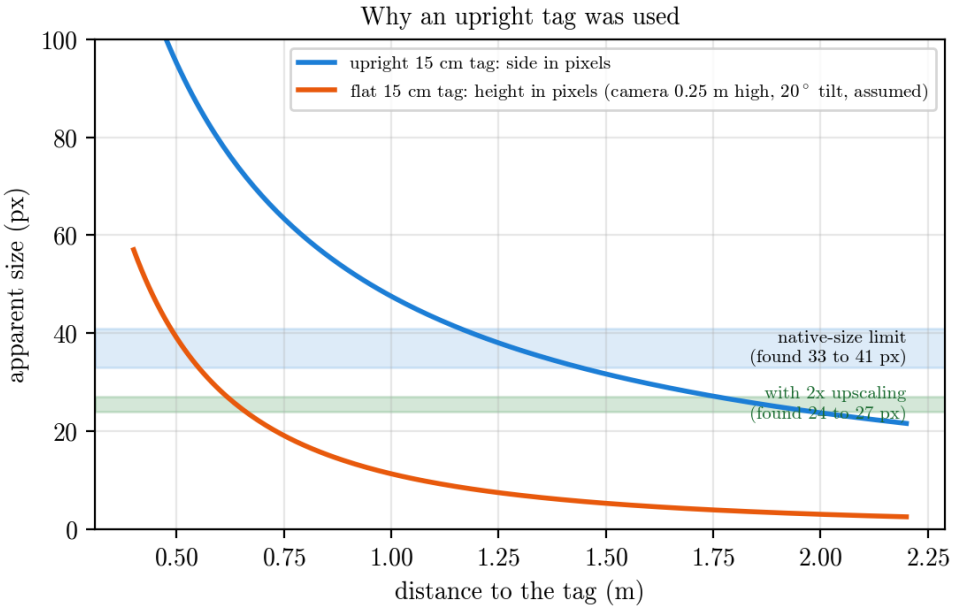


Figure 6: Apparent size of a 15 cm tag against distance for an upright and a flat tag, with the measured detection limits. The flat-tag curve uses an assumed camera height of 0.25 m.

(ii) Search and centering finite state machine

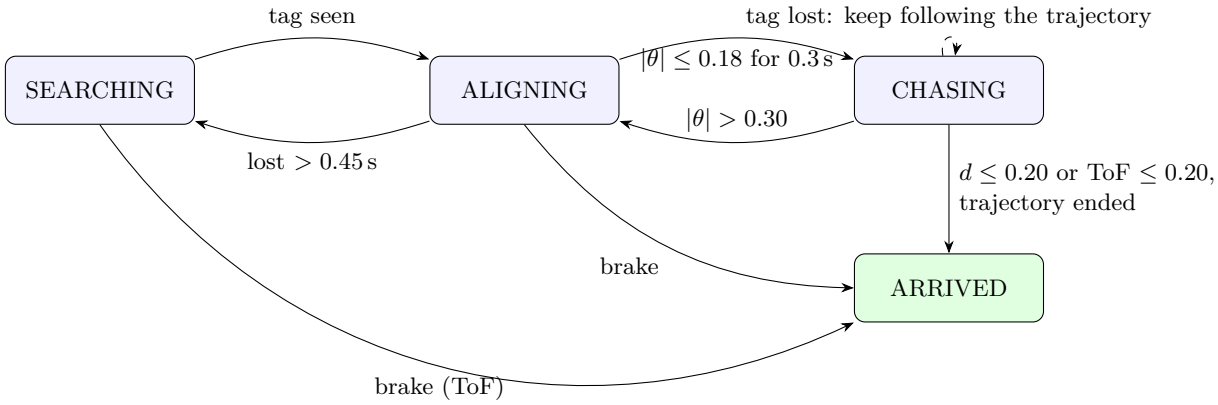


Figure 7: State machine of `state_machine.py`. The brake test is made first in every control tick.

Parameter	Value	Meaning
`lost_timeout`	0.45 s	no tag for longer than this: SEARCHING
`search_rate`	0.30 rad/s	in-place rotation, toward the side where the tag was last seen
`align_tol`	0.18 rad (10.3°)	tag centred: start chasing
`settle_time`	0.3 s	bearing must stay inside the tolerance this long
`align_gain`, limits	0.8 s^{-1}; 0.12 to 0.40 rad/s	$\omega_z = \mathrm{sat}(K\theta)$ while aligning
`realign_tol`	0.30 rad	hysteresis: leave CHASING only beyond this
`chase_bearing_gain`	0.8 s^{-1}	small correction $K\theta$ added to the flat-output ω
`brake_distance`	0.20 m	camera range or ToF threshold
`blind_chase`, range	true, 1.8 m	continue the trajectory if the tag leaves the image

Sign convention. The bearing $\theta > 0$ means the tag is to the *left* of the camera axis (ROS REP-103, $+\omega_z$ counter-clockwise), so aligning is $\omega_z = +K\theta$. In SEARCHING the direction of the sweep is the side on which the tag was last seen, and counter-clockwise if it has never been seen.

Why align in place before translating. The planned path is a straight cubic in the robot frame (the boundary conditions require $y(0) = y(T) = 0$), so it assumes the tag lies straight ahead. If the robot drove while its heading was off by θ, the open-loop path would end laterally displaced by about $d \sin\theta$: for the real run ($d = 1.28\,\text{m}$) a 10° error already gives $1.28 \sin 10° = 22\,\text{cm}$, which is larger than the tag itself. The tag would also drift toward the edge of the image (the half field of view is about 45°), where it is partly clipped or lost. Rotating first removes the heading error at zero translation, so the planned and executed paths coincide and the tag stays near the image centre. Both real runs show this: after the first two seconds of the chase the logged bearing error stayed within 0.02 rad.

(iii) Standoff braking

The brake is the first test in every control tick, before any other state logic: if the camera ground distance is $d \le 0.20\,\text{m}$ or the front ToF reads $\le 0.20\,\text{m}$, the state becomes ARRIVED and $(v, \omega) = (0, 0)$ is commanded immediately and the state is latched. This behaviour is covered by unit tests (`test_fsm_brake_on_camera_and_tof`). **On the robot neither sensor brake was triggered**: in both runs the upright tag left the top of the image at about 0.35 to 0.38 m, and the ToF never returned a reading below its maximum range, so the stop came from the end of the planned trajectory (state `ARRIVED (trajectory end, tag out of view)`), which is designed to end exactly at the standoff. The measured gap of 20 cm confirms that this worked in these two runs.

Video stalls. The runs used the driver's own decoder and the published topic `camera/image_color`. The driver log contains no decoder or buffer errors (its only warning is that the optional FFmpeg transport package is not installed). The optional direct-capture mode of the node (`camera_source:=cv2`) implements the recommended grab thread with a `time.sleep(0.010)` between `cap.grab()` calls; it was not needed and was not used.

Problem 2: Differential flatness trajectory generation

(i) Flatness inversion

For a unicycle $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$, the position $z(t) = [x(t), y(t)]^T$ is a flat output: the speed is $v = \sqrt{\dot{x}^2 + \dot{y}^2}$ and the heading is $\theta = \text{atan2}(\dot{y}, \dot{x})$, so

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}, \qquad \omega = \dot{\theta} = \frac{d}{dt}\,\text{atan2}(\dot{y}, \dot{x}) = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}. \tag{1}$$

At standstill ($\dot{x}^2 + \dot{y}^2 \to 0$) the heading is undefined; the implementation returns $\omega = 0$ there. The formula was verified on a circle of radius R traversed at angular rate Ω (it returns $v = R\Omega$ and $\omega = \Omega$; unit test `test_flat_inversion_circle`).

(ii) Cubic polynomial matrix solution

Each component is $z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]} t + \alpha_2^{[j]} t^2 + \alpha_3^{[j]} t^3$ ($j \in \{x, y\}$) over the horizon T. Imposing $z_j(0)$, $\dot{z}_j(0)$, $z_j(T)$, $\dot{z}_j(T)$ gives the linear system

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0^{[j]} \\ \alpha_1^{[j]} \\ \alpha_2^{[j]} \\ \alpha_3^{[j]} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix}, \tag{2}$$

which `solve_flat_polynomial(z0, zdot0, zf, zdotf, T)` solves with `numpy.linalg.solve`. With the boundary values of the assignment, $x(0) = 0$, $\dot{x}(0) = 0.06\,\text{m/s}$, $x(T) = d_{\text{tag}} - 0.20\,\text{m}$, $\dot{x}(T) = 0$ and $y(0) = \dot{y}(0) = y(T) = \dot{y}(T) = 0$, the closed form is

$$\alpha_0 = 0, \quad \alpha_1 = \dot{z}_0, \quad \alpha_2 = \frac{3z_f - 2\dot{z}_0 T}{T^2}, \quad \alpha_3 = \frac{\dot{z}_0 T - 2z_f}{T^3} \qquad (\dot{z}_f = 0),$$

(obtained by solving the last two rows of Eq. (2) for α_2 and α_3). For y all four boundary values are zero, hence $\alpha^{[y]} = \mathbf{0}$, $y(t) \equiv 0$ and, by Eq. (1), $\omega(t) \equiv 0$: the flat path is a straight line in the robot frame and the speed profile is $v(t) = \dot{x}(t) = \alpha_1 + 2\alpha_2 t + 3\alpha_3 t^2$. The turning during the chase comes only from the small bearing correction.

(iii) Time horizon scaling

$$T = \max\left(4.0,\ \frac{1.6\,(d_0 - 0.20)}{v_{\max}}\right), \qquad v_{\max} = 0.16\ \text{m/s}. \tag{3}$$

The mean speed of the path is $z_f/T = (d_0 - 0.20)/T = v_{\max}/1.6 = 0.10\,\text{m/s}$, and a cubic that starts slowly and ends at rest peaks at roughly 1.4 times its mean. Because T scales with the distance, the peak is the same for every d_0 (the 4 s floor only matters for starts closer than $0.2 + 4 \cdot 0.16/1.6 = 0.6\,\text{m}$). The coefficient table below was computed with the package; the peak is 0.137 m/s in all cases, below 0.16 m/s, and $v(T) = 0$. In the real runs, $d_0 = 1.37\,\text{m}$ (run 3) and 1.28 m (run 4) give $T = 11.7\,\text{s}$ and 10.8 s, and the chases lasted 11.8 s and 10.9 s (the extra 0.1 to 0.2 s is the control-tick granularity).

d_0 (m)	T (s)	$\alpha_2^{[x]}$ (m/s^2)	$\alpha_3^{[x]}$ (m/s^3)	peak v (m/s)	at t (s)
0.90	7.00	0.02571	-28.571×10^{-4}	0.137	3.00
1.00	8.00	0.02250	-21.875×10^{-4}	0.137	3.43
1.28	10.80	0.01667	-12.003×10^{-4}	0.137	4.63
1.50	13.00	0.01385	-8.284×10^{-4}	0.137	5.57
1.80	16.00	0.01125	-5.469×10^{-4}	0.137	6.86

Trajectory coefficients. Cubic coefficients of $x(t)$ and peak speed for several start distances, computed with `FlatTrajectory`.

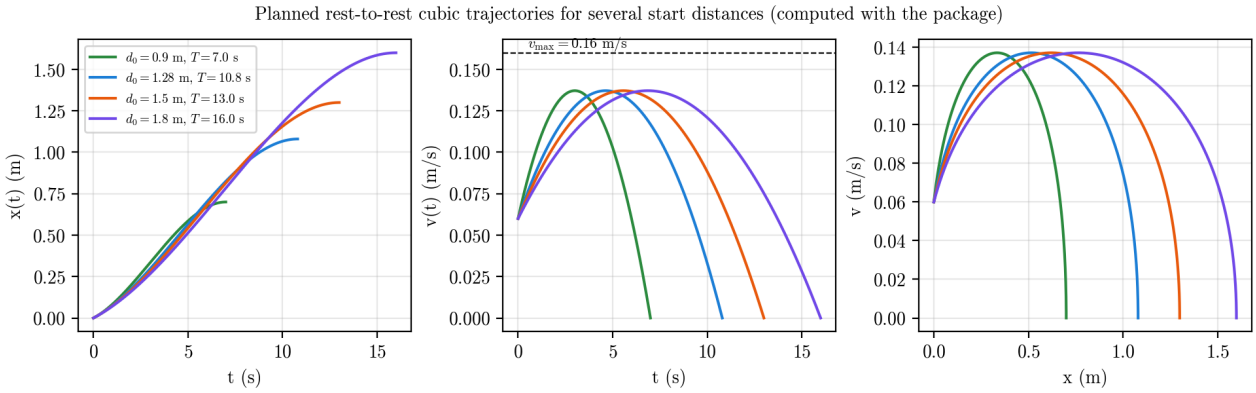


Figure 8: Planned $x(t)$, $v(t)$ and $v(x)$ for several start distances.

Problem 3: Motor voltage telemetry and dynamics logging

(i) Kinematics to wheel rates

With $r = 0.05\,\text{m}$ and $L_c = 0.20\,\text{m}$, Eq. (4) of the assignment is implemented as

$$\omega_1 = \omega_4 = \frac{v_x + L_c\omega_z}{r}, \qquad \omega_2 = \omega_3 = \frac{v_x - L_c\omega_z}{r} \tag{4}$$

(wheel 1 is front-right, 2 front-left, 3 rear-left, 4 rear-right). Numerical examples that appear in the logs: the in-place search at $\omega_z = 0.30\,\text{rad/s}$ gives $\omega = \pm 1.2\,\text{rad/s}$; the peak cruising speed $v_x = 0.137\,\text{m/s}$ gives $\omega = 2.74\,\text{rad/s}$ ($= 26.2\,\text{rpm}$) on all four wheels.

(ii) Motor armature voltage model

$$V_i(t) = K_v\,\omega_i(t) + K_a\,\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t}, \qquad K_v = 0.38\ \text{V s/rad}, \quad K_a = 0.06\ \text{V s}^2/\text{rad}. \tag{5}$$

It is evaluated in every control tick ($\Delta t = 0.05\,\text{s}$) on the wheel rates computed from the *commanded* (v_x, ω_z). The voltages are therefore model outputs: the motor controller's own voltage is not measured. The first tick uses a zero acceleration term, and once the robot has halted the wheel rates are zero for two consecutive ticks, so $V_i = 0$ exactly (no acceleration term left). Figure 9 splits the logged V_1 of run 4 into its two terms: the back-EMF term $K_v\omega_1$ carries the whole cruise (up to 1.05 V), and the inertia term $K_a\Delta\omega_1/\Delta t$ is nonzero only at the velocity steps (starting to turn in SEARCHING, the switch from ALIGNING to CHASING, where the speed jumps to the initial 0.06 m/s, and the final deceleration), reaching about $+1.4\,\text{V}$ at the start of the chase and $-1.3\,\text{V}$ at an in-place reversal.

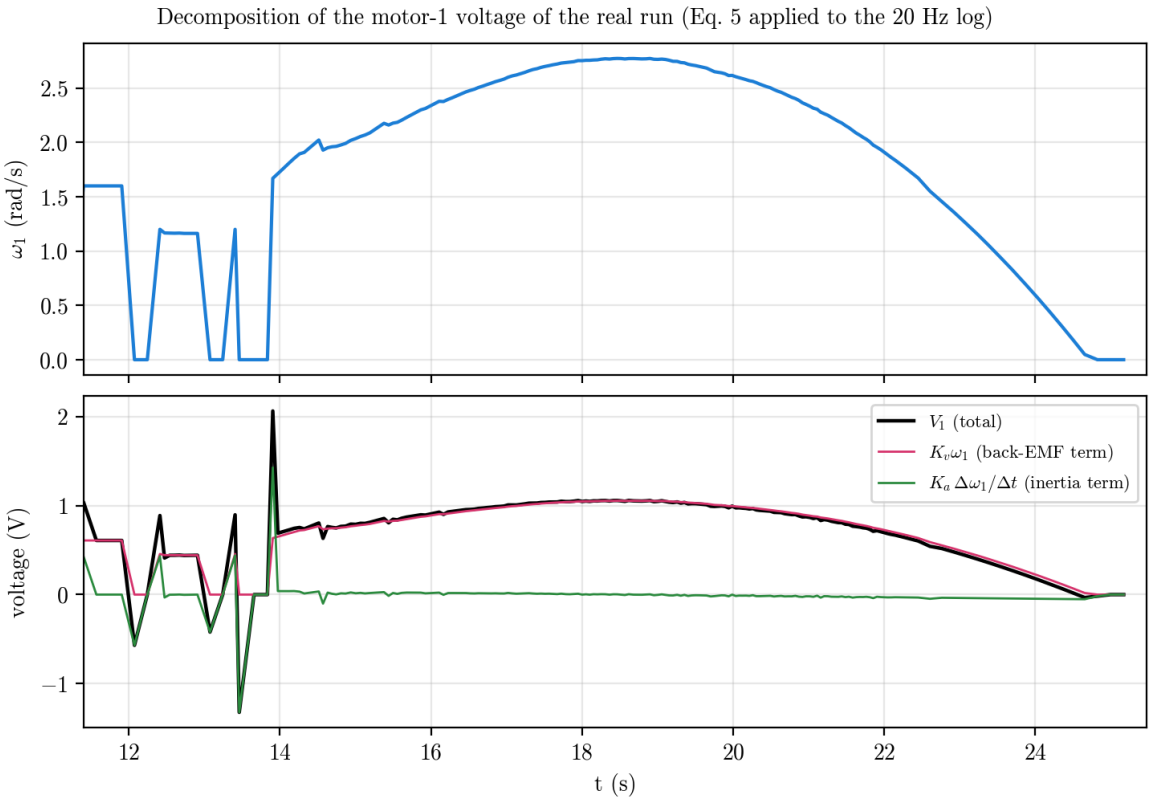


Figure 9: Back-EMF and inertia terms of V_1 in the real run (20 Hz log).

(iii) Live table and CSV logging

The node prints one table line per second and writes the same values to `motor_voltage_log.csv` (columns: `t_s`, `V1_V..V4_V`, `state`, `v_mps`, `w_radps`, `distance_m`, `bearing_rad`, `tof_m`, `w1..w4_radps`); a 20 Hz copy (`motor_voltage_fast.csv`) is used for the plots. When the state becomes ARRIVED the node waits three control ticks and then writes the last row at the true stop time, so that the voltages in that row are exactly zero.

flatness_nav console output (rendered from motor_voltage_log.csv)

```
ros2 launch rmi_flatness_av flatness_av.launch.py tag_id:=3 tag_size:=0.15 csv_path:=PWD/motor_voltage_log.csv
[INFO] [flatness_nav]: Tag tag36h11 id=3 size=0.15 m. Table header: t(s) V1 V2 V3 V4 state. CSV -> .../motor_voltage_log.…
[INFO] [flatness_nav]:     0 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     1 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     2 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     3 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     4 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     5 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     6 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     7 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     8 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:     9 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:    10 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:    11 s | V1..V4 =   0.456  -0.456  -0.456   0.456 V | SEARCHING
[INFO] [flatness_nav]:    12 s | V1..V4 =  -0.569   0.569   0.569  -0.569 V | ALIGNING
[INFO] [flatness_nav]:    13 s | V1..V4 =  -0.420   0.420   0.420  -0.420 V | ALIGNING
[INFO] [flatness_nav]:    14 s | V1..V4 =   0.707   0.352   0.352   0.707 V | CHASING
[INFO] [flatness_nav]:    15 s | V1..V4 =   0.789   0.675   0.675   0.789 V | CHASING
[INFO] [flatness_nav]:    16 s | V1..V4 =   0.926   0.880   0.880   0.926 V | CHASING
[INFO] [flatness_nav]:    17 s | V1..V4 =   1.007   0.979   0.979   1.007 V | CHASING
[INFO] [flatness_nav]:    18 s | V1..V4 =   1.046   1.033   1.033   1.046 V | CHASING
[INFO] [flatness_nav]:    19 s | V1..V4 =   1.048   1.017   1.017   1.048 V | CHASING
[INFO] [flatness_nav]:    20 s | V1..V4 =   0.975   0.962   0.962   0.975 V | CHASING
[INFO] [flatness_nav]:    21 s | V1..V4 =   0.867   0.839   0.839   0.867 V | CHASING
[INFO] [flatness_nav]:    22 s | V1..V4 =   0.686   0.666   0.666   0.686 V | CHASING
[INFO] [flatness_nav]:    23 s | V1..V4 =   0.440   0.440   0.440   0.440 V | CHASING
[INFO] [flatness_nav]:    24 s | V1..V4 =   0.169   0.169   0.169   0.169 V | CHASING
[INFO] [flatness_nav]:  25.2 s | V1..V4 =   0.000   0.000   0.000   0.000 V | ARRIVED (trajectory end, tag out of view)
[INFO] [flatness_nav]: Halted at 25.2 s (trajectory end, tag out of view); log saved
```

Figure 10: Console output of the node for run 4, rendered from `motor_voltage_log.csv` in the node's log format (the console text itself was not saved during the run; the CSV is the real data).

Voltage log of run 4. The table below is Table 1 of the assignment, filled with the real values (one row per second).

Table 1. Chassis motor armature voltage log (target: Tag 3), from `motor_voltage_log.csv`, one row per second.

t (s)	V_1 (V)	V_2 (V)	V_3 (V)	V_4 (V)	**State**
0	0.456	-0.456	-0.456	0.456	`SEARCHING`
1	0.456	-0.456	-0.456	0.456	`SEARCHING`
2	0.456	-0.456	-0.456	0.456	`SEARCHING`
3	0.456	-0.456	-0.456	0.456	`SEARCHING`
4	0.456	-0.456	-0.456	0.456	`SEARCHING`
5	0.456	-0.456	-0.456	0.456	`SEARCHING`
6	0.456	-0.456	-0.456	0.456	`SEARCHING`
7	0.456	-0.456	-0.456	0.456	`SEARCHING`
8	0.608	-0.608	-0.608	0.608	`ALIGNING`
9	0.508	-0.508	-0.508	0.508	`ALIGNING`
10	0.645	0.480	0.480	0.645	`CHASING`
11	0.835	0.795	0.795	0.835	`CHASING`
12	0.969	0.960	0.960	0.969	`CHASING`
13	1.041	1.036	1.036	1.041	`CHASING`
14	1.043	1.009	1.009	1.043	`CHASING`
15	0.927	0.922	0.922	0.927	`CHASING`
16	0.737	0.737	0.737	0.737	`CHASING`
17	0.460	0.460	0.460	0.460	`CHASING`
18	0.087	0.087	0.087	0.087	`CHASING`
T_{stop}=18.9	0.000	0.000	0.000	0.000	`ARRIVED (trajectory end, tag out of view)`

Run description (tag `tag36h11`, ID 3, black square 0.15 m, standing upright facing the robot; 360 p video; wired USB link; `motor_voltage_log.csv`).

- **SEARCHING** from 0 to 11.4 s: the robot started facing away from the tag and swept in place at 0.30 rad/s, so $V_1 = V_4 = +0.456$ V and $V_2 = V_3 = -0.456$ V.
- **ALIGNING** from 11.4 to 13.9 s: the tag came into view and the robot rotated in place (alternating voltage signs while it corrected) until $|\theta| \leq 0.18$ rad, then settled for 0.3 s.
- **CHASING** from 13.9 to 24.8 s: the cubic trajectory was planned from $d_0 \approx 1.28$ m (bearing 0.15 rad), giving $T = 1.6(1.28 - 0.20)/0.16 = 10.8$ s, and the chase took 10.9 s. The peak speed was 0.137 m/s (limit 0.16). The measured camera distance fell from 1.280 m to 0.351 m (at $t = 22$ s) with a bearing error of about 0.01 rad.
- **ARRIVED** at 24.8 s: below about 0.35 m the upright tag left the top of the image (its centre is above the optical axis), so the last part of the approach ran from the planned trajectory alone, which ends at the 20 cm standoff. The state was logged as `ARRIVED (trajectory end, tag out of view)`. The row at $T_{\text{stop}} = 25.2$ s shows $V_1 = V_2 = V_3 = V_4 = 0.000$ V, as required.

Measured stop distance: for the run documented above, a tape measure gave a gap of **20 cm** between the front of the robot and the tag after it halted, equal to the target. An earlier trial with the same tag and code (a longer 36 s search) also gave 20 cm. Measurements are by hand (resolution about 1 cm) and the stop is reached by the dead-reckoned trajectory, so it depends on the distance estimate taken at the start of the chase.

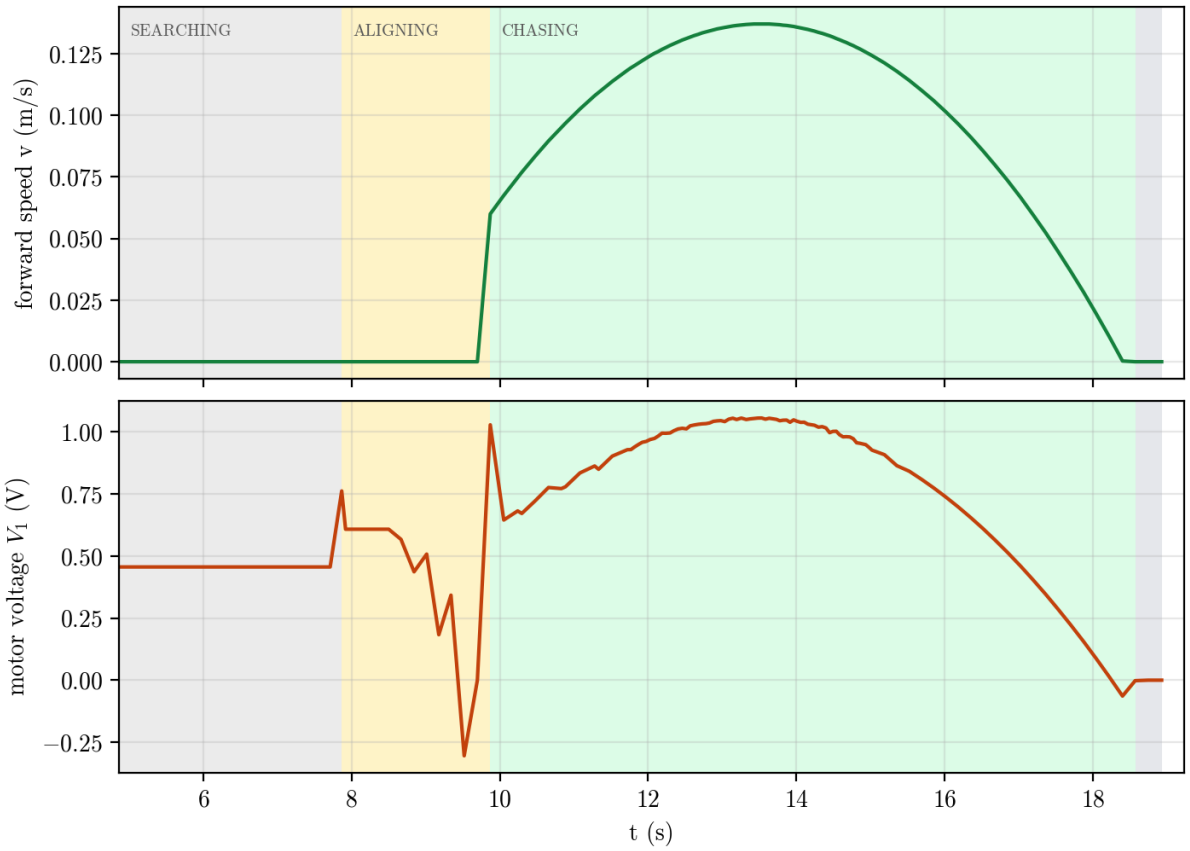


Figure 11: Forward speed $v(t)$ and motor voltage $V_1(t)$ during the Tag 3 run. The first part of the search phase (constant $v = 0$ and $V_1 = 0.456\,\mathrm{V}$) is omitted from the axis, as stated in the axis label.

Repeatability. Aligned at the start of the chase, the two successful runs are almost the same (Figure 12): the peak speed is 0.137 m/s in both, and the chase lasted 11.8 s and 10.9 s for start distances of 1.37 m and 1.28 m. The run-history table in Section 1.4 lists all four runs.

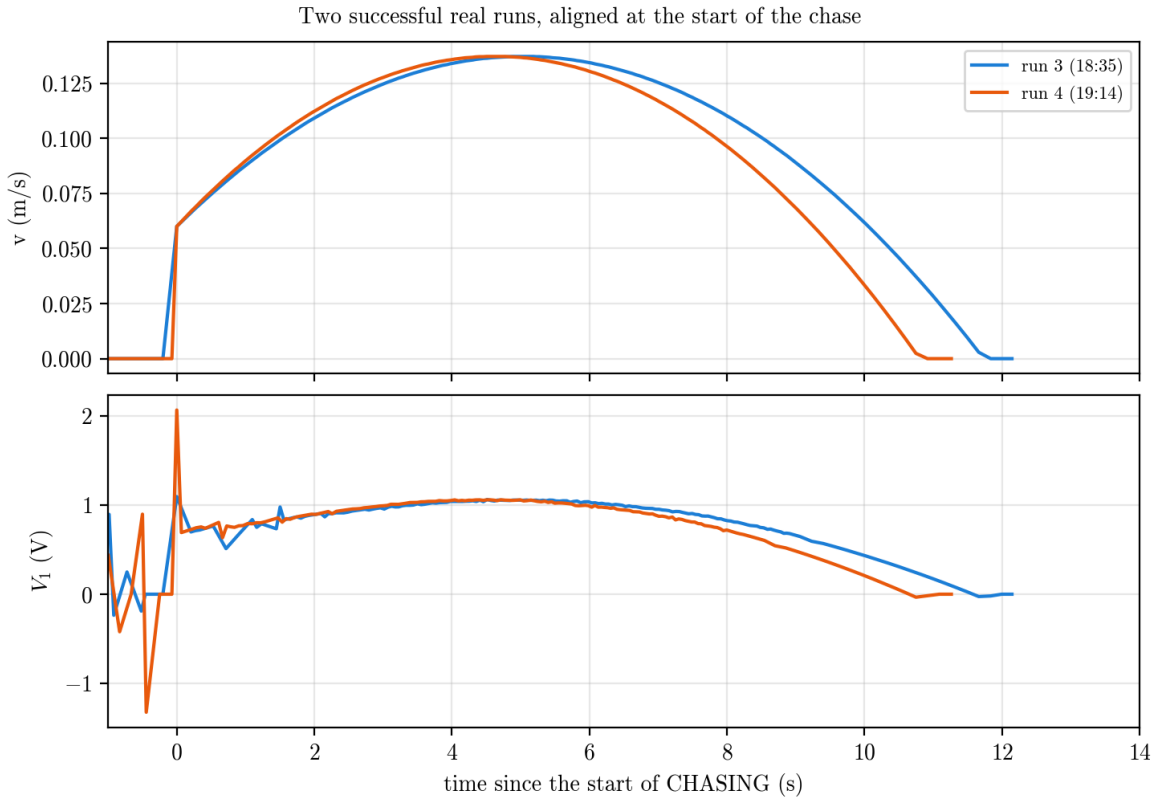


Figure 12: Runs 3 and 4 aligned at the start of CHASING (real data, 20 Hz log).

Verification

Unit tests. 18 tests cover the boundary conditions of Eq. (2), the horizon formula and its 4 s floor, zero turn rate and the speed limit of the straight trajectory, Eq. (1) on a circle and at standstill, Eq. (4) and Eq. (5) values, every state transition (including the brake, loss before the chase, and continuing after loss during the chase), and the detector's recovery of range and bearing from rendered tags with the real camera intrinsics.

```
ameen@ameen-G6: ~/ws  (unit tests)

$ python -m pytest -v src/rm_flatness_nav/test
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /home/ameen/rm_venv/bin/python
rootdir: /home/ameen/robomasters/assignment2/ws/src/rm_flatness_nav
plugins: colcon-core-0.21.3, cov-4.1.0
collecting ... collected 18 items

test/test_core.py::test_polynomial_meets_boundary_conditions PASSED [  5%]
test/test_core.py::test_time_horizon_formula_and_floor PASSED [ 11%]
test/test_core.py::test_straight_trajectory_has_zero_turn_rate_and_limited_speed PASSED [ 16%]
test/test_core.py::test_flat_inversion_circle PASSED [ 22%]
test/test_core.py::test_flat_inversion_standstill_is_finite PASSED [ 27%]
test/test_core.py::test_wheel_rates_eq4 PASSED       [ 33%]
test/test_core.py::test_voltage_model_eq5 PASSED     [ 38%]
test/test_core.py::test_fsm_search_turns_toward_last_seen_side PASSED [ 44%]
test/test_core.py::test_fsm_aligns_in_place_before_chasing PASSED [ 50%]
test/test_core.py::test_fsm_brake_on_camera_and_tof PASSED [ 55%]
test/test_core.py::test_fsm_chase_speed_within_limit PASSED [ 61%]
test/test_core.py::test_fsm_loss_before_chase_returns_to_search PASSED [ 66%]
test/test_core.py::test_fsm_tag_leaving_image_during_chase_keeps_following_trajectory PASSED [ 72%]
test/test_core.py::test_fsm_blind_chase_can_be_disabled PASSED [ 77%]
test/test_detector.py::test_detects_and_recovers_ground_range_and_bearing[0.7-0.0] PASSED [ 83%]
test/test_detector.py::test_detects_and_recovers_ground_range_and_bearing[0.7-0.1] PASSED [ 88%]
test/test_detector.py::test_detects_and_recovers_ground_range_and_bearing[0.7--0.1] PASSED [ 94%]
test/test_detector.py::test_ignores_other_tag_ids PASSED [100%]

============================== 18 passed in 0.71s ==============================
```

Figure 13: Output of `pytest -v` (real, run on the host).

```
ameen@ameen-G6: ~/ws  (colcon build)

$ cd ws && colcon build --symlink-install --packages-select rm_flatness_nav
Starting >>> rm_flatness_nav
Finished <<< rm_flatness_nav [4.14s]

Summary: 1 package finished [4.48s]
```

Figure 14: The package builds with `colcon build -symlink-install`.

Offline simulation (not hardware data). The same state machine, trajectory and voltage model were simulated against a point-robot model with a tag that becomes invisible inside a minimum range, to exercise cases that were not run on the robot (different starting headings). All three scenarios halt at the standoff.

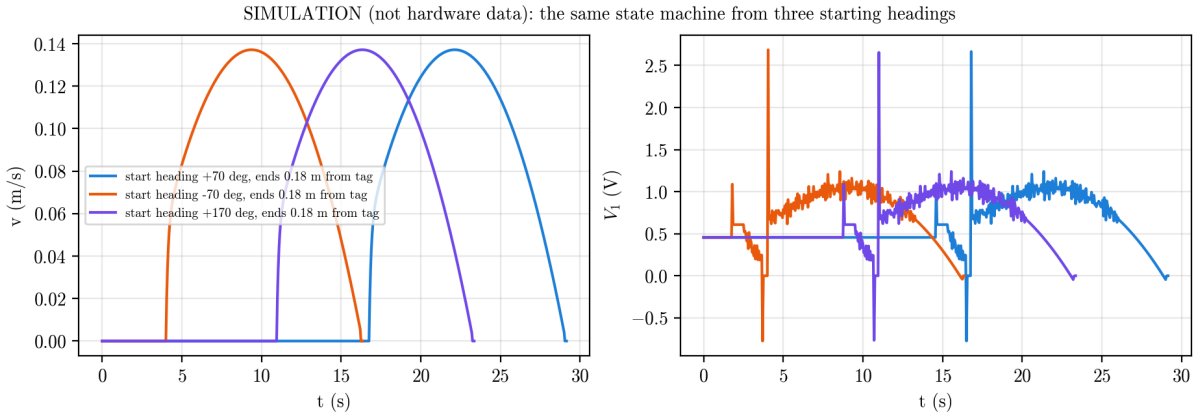


Figure 15: Simulation only: three starting headings.

```
ameen@ameen-G6: ~/ws  (offline simulation, not hardware data)

$ python -m rm_flatness_nav.sim --distance 1.0 --lateral 0.1 --heading-deg 60 --detect-range 1.2 --min-range 0.6   # …
SIMULATION: final state ARRIVED (trajectory end, tag out of view), distance to tag 0.183 m
         0       0.456      -0.456      -0.456       0.456   SEARCHING
         1       0.456      -0.456      -0.456       0.456   SEARCHING
         2       0.456      -0.456      -0.456       0.456   SEARCHING
         3       0.456      -0.456      -0.456       0.456   SEARCHING
         4       0.456      -0.456      -0.456       0.456   SEARCHING
         5       0.456      -0.456      -0.456       0.456   SEARCHING
       ...         ...         ...         ...         ...         ...
        25       0.236       0.236       0.236       0.236     CHASING
      25.9       0.000       0.000       0.000       0.000  ARRIVED (trajectory end, tag out of view)
```

Figure 16: Output of `python -m rm_flatness_nav.sim` (simulation, not hardware data).

Trials with the three printed tags lying flat

After the runs above, all three printed tags (`tag36h11`, IDs 1, 2, 3, black square 150 mm) were laid flat on the carpet, as the assignment describes, with Tag 3 in the middle and Tags 1 and 2 about 0.3 m to its left and right as distractors. The node follows only ID 3.

Static detection test (real camera frames). With the row 0.6 m in front of the robot, the project's own detector (native scale) found all three tags: Tag 1 83×27 px, Tag 3 69×27 px, Tag 2 84×28 px (Figure 17). With the row moved to about 0.8 m only Tag 2 was found, at 47×9 px, and Tag 3 was missed; at about 1 m nothing was found. So in a single frame a flat tag is readable out to roughly 0.6 to 0.7 m with this camera. The node's search is more tolerant (it also tries a 2× enlarged image and sees the tag over several frames).

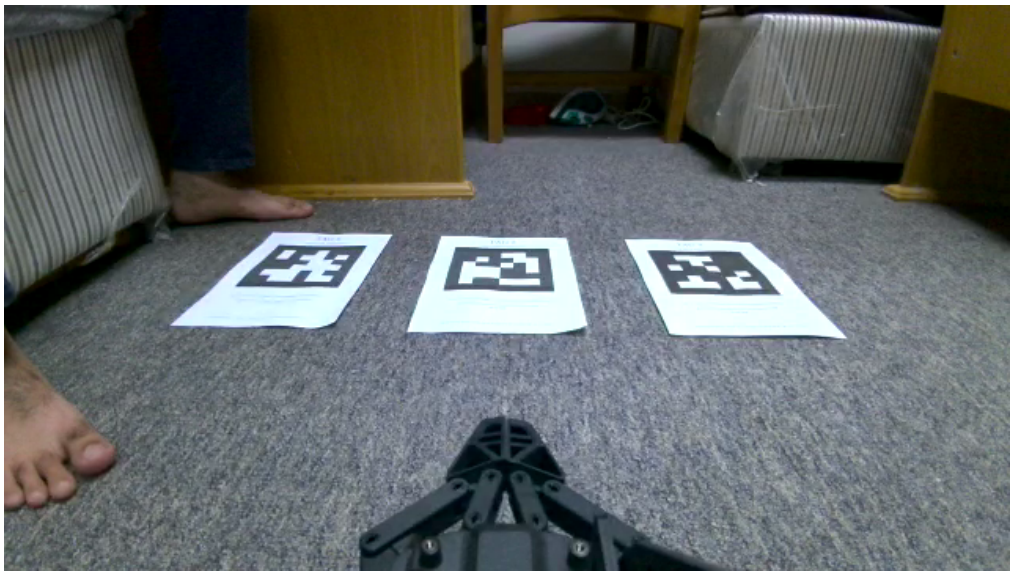


Figure 17: Real frame from the robot: the three printed tags lying flat, 0.6 m ahead. All three are detected (IDs 1, 3, 2 from left to right).

Mission attempts. Three autonomous attempts were made with this layout (about 1 m to Tag 3). **None reached ARRIVED, so none is used in Table 1 or the plots.** In the first the tag was not detected during a 33 s sweep. In the other two it was detected, the robot aligned and began to chase (for about 8 s in the last one), then lost the tag and returned to SEARCHING. In both the logged distances were about 100 times too large (for example 95.8 where the tag was about 1 m away), which is what a tag size typed as 15 instead of 0.15 would produce; because of this scale error the rule that continues the trajectory when the tag leaves the image (it only applies within 1.8 m) could not act. This cause is inferred from the numbers; the node's start-up line with the tag size was not kept. The tag-size prompt of the helper script now rejects implausible values. The logs are kept in `logs/flat_failed_attempts/`. A clean flat-tag run with the correct tag size was not completed, so the evidence for the full mission remains runs 3 and 4 (upright tag).

Deviations from the assignment text and limitations

- **Upright tag.** The assignment places the tag *flat* on the floor; the tag stood upright, facing the robot, because a flat 15 cm tag is a thin sliver for this camera (Figure 6). A later trial with flat tags (Section on flat tags) confirmed that a flat tag is readable only out to about 0.6 to 0.7 m in a single frame, and no flat-tag mission completed.
- **Distance.** The assignment gives 1.5 to 1.8 m. The tag was about 1.3 to 1.4 m away (the robot only rotates before the chase, so the camera range at the first detection is the starting distance: 1.375 m in run 3 and 1.28 m in run 4). The detector could see further with its upscaled search, but the tag was not placed there.
- **Camera and arm.** The arm pose did not change the camera view; the camera tilt of about 20° was read from images.
- **Stop source.** The stop came from the planned trajectory, not from a camera or ToF reading. The sensor brakes are implemented and unit-tested but were not triggered on the robot.
- **Voltages** are model values computed from commanded velocities (Eq. (5)); they are not measured at the motors.
- **Number of trials.** Two successful runs; the stop gap was measured by hand to about 1 cm.
- **Tags 1 and 2.** The successful runs (3 and 4) used Tag 3 alone. Tags 1 and 2 were placed beside it only in the flat-tag trials, where they were detected but ignored by ID; no successful mission

with all three tags was logged.

Deliverables checklist

Item	Status	Where
Equations for flatness inversion and the boundary matrix solution	done	Eq. (1), Eq. (2)
Completed Table 1 with experimental voltages	done	Section 4(iii)
Plot of $v(t)$ and $V_1(t)$	done	Section 4(iii)
Code archive: package `rm_flatness_nav` and `motor_voltage_log.csv`	done	`assignment2_submission.zip`, repository
Demonstration video ($\leq$ 45 s): sweeping, centering, cruising and stopping at 20 cm	to be added	**[ADD VIDEO LINK HERE]** (edit the box at the top of the source)

Source listings

`flatness.py`

```
"""Differential-flatness trajectory generation for a unicycle-type robot.

The flat output is the planar position z(t) = [x(t), y(t)]^T.  Every smooth curve
for z(t) gives the linear speed v and turn rate w in closed form (Eq. 1), so no
numerical optimiser is needed.  Each component is a cubic over horizon T (Eq. 2).
"""
import math

import numpy as np

EPS = 1e-6
V_MAX = 0.16        # m/s cruise limit (Eq. 3)
STANDOFF = 0.20     # m, final distance from the tag
T_MIN = 4.0         # s
T_GAIN = 1.6


def flat_inversion(xd, yd, xdd, ydd):
    """Eq. (1): recover (v, w) from the first and second derivatives of z(t)."""
    speed_sq = xd * xd + yd * yd
    v = math.sqrt(speed_sq)
    if speed_sq < EPS:
        # heading is undefined at standstill (0/0): command no turning
        return v, 0.0
    w = (xd * ydd - yd * xdd) / speed_sq
    return v, w


def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    """Eq. (2): cubic coefficients [a0, a1, a2, a3] of z(t) matching the four
    boundary values z(0), z'(0), z(T), z'(T).  Solved with numpy.linalg.solve."""
    A = np.array([[1.0, 0.0, 0.0, 0.0],
                  [0.0, 1.0, 0.0, 0.0],
                  [1.0, T, T ** 2, T ** 3],
                  [0.0, 1.0, 2.0 * T, 3.0 * T ** 2]])
    b = np.array([z0, zdot0, zf, zdotf], dtype=float)
    return np.linalg.solve(A, b)


def eval_cubic(alpha, t):
    """Position, velocity and acceleration of the cubic at time t."""
    a0, a1, a2, a3 = alpha
    return (a0 + a1 * t + a2 * t ** 2 + a3 * t ** 3,
            a1 + 2.0 * a2 * t + 3.0 * a3 * t ** 2,
            2.0 * a2 + 6.0 * a3 * t)


def time_horizon(d0, standoff=STANDOFF, v_max=V_MAX, t_min=T_MIN, gain=T_GAIN):
    """Eq. (3): T = max(4.0, 1.6 (d0 - 0.20) / v_max)."""
    return max(t_min, gain * (d0 - standoff) / v_max)


class FlatTrajectory:
    """Straight-ahead cubic pursuit in the robot frame at the moment of centering.

    Boundary conditions (assignment, Problem 2 ii):
        x(0)=0, x'(0)=0.06 m/s, x(T)=d0-0.20, x'(T)=0;  y(0)=0, y'(0)=0, y(T)=0, y'(T)=0.
    """

    def __init__(self, d0, v0=0.06, standoff=STANDOFF, v_max=V_MAX):
        self.d0 = d0
        self.T = time_horizon(d0, standoff, v_max)
        goal = max(d0 - standoff, 0.0)
        self.alpha_x = solve_flat_polynomial(0.0, v0, goal, 0.0, self.T)
        self.alpha_y = solve_flat_polynomial(0.0, 0.0, 0.0, 0.0, self.T)
```

```
66
67      def sample(self, t):
68          """(v, w, x, y) at time t since the trajectory started; clamps to [0, T]."""
69          t = min(max(t, 0.0), self.T)
70          x, xd, xdd = eval_cubic(self.alpha_x, t)
71          y, yd, ydd = eval_cubic(self.alpha_y, t)
72          v, w = flat_inversion(xd, yd, xdd, ydd)
73          # the cubic may undershoot slightly below zero speed near T; never reverse
74          if xd < 0.0:
75              v = 0.0
76          return v, w, x, y
77
78      def finished(self, t):
79          return t >= self.T
```

wheel_voltage.py

```
1  """Wheel-rate kinematics (Eq. 4) and armature voltage model (Eq. 5) for the RoboMaster EP."""
2
3  WHEEL_RADIUS = 0.05   # r, m
4  CHASSIS_RADIUS = 0.20  # L_c, m
5  K_V = 0.38            # V.s/rad  (back-EMF)
6  K_A = 0.06            # V.s^2/rad (inertia)
7
8
9  def wheel_rates(vx, wz, r=WHEEL_RADIUS, lc=CHASSIS_RADIUS):
10     """Eq. (4): returns (w1, w2, w3, w4) = (w_FR, w_FL, w_RL, w_RR) in rad/s."""
11     w1 = (vx + lc * wz) / r   # front-right
12     w2 = (vx - lc * wz) / r   # front-left
13     return (w1, w2, w2, w1)   # w3 = w_RL = w2, w4 = w_RR = w1
14
15
16 class MotorVoltageModel:
17     """Eq. (5): V_i = K_v w_i + K_a (w_i(t) - w_i(t-dt)) / dt, per motor."""
18
19     def __init__(self, kv=K_V, ka=K_A):
20         self.kv = kv
21         self.ka = ka
22         self._prev = None
23
24     def reset(self):
25         self._prev = None
26
27     def step(self, omegas, dt):
28         """Voltages for the current wheel rates; dt is the time since the last call."""
29         if self._prev is None or dt <= 0.0:
30             accel = (0.0,) * len(omegas)
31         else:
32             accel = tuple((w - p) / dt for w, p in zip(omegas, self._prev))
33         self._prev = tuple(omegas)
34         return tuple(self.kv * w + self.ka * a for w, a in zip(omegas, accel))
```

state_machine.py

```
1  """Supervisory state machine: SEARCHING -> ALIGNING -> CHASING -> ARRIVED.
2
3  Pure logic (no ROS), driven by `update()` once per control tick, so it can be unit
4  tested and simulated.  Sign convention: bearing > 0 means the tag is to the LEFT of the
5  camera axis (ROS REP-103, +w = counter-clockwise), so aligning is w = +K * bearing.
6  """
7  from dataclasses import dataclass
8  from typing import Optional
9
10 from .flatness import FlatTrajectory
11
12 SEARCHING, ALIGNING, CHASING, ARRIVED = 'SEARCHING', 'ALIGNING', 'CHASING', 'ARRIVED'
13
14
15 @dataclass
16 class Params:
17     lost_timeout: float = 0.45       # s without a tag -> SEARCHING
18     search_rate: float = 0.30        # rad/s, in-place sweep
19     align_tol: float = 0.18          # rad (about 10 deg) to start chasing
20     realign_tol: float = 0.30        # rad, hysteresis: leave CHASING only beyond this
21     settle_time: float = 0.30        # s the bearing must stay inside align_tol
22     align_gain: float = 0.8          # 1/s
23     align_min_rate: float = 0.12     # rad/s, overcome wheel dead-band
24     align_max_rate: float = 0.40     # rad/s
25     chase_bearing_gain: float = 0.8  # 1/s, small correction added to the flat-output w
26     chase_max_w: float = 0.40        # rad/s
27     brake_distance: float = 0.20     # m, camera distance or ToF threshold
28     v_max: float = 0.16              # m/s
29     blind_chase: bool = True         # keep following the trajectory if the tag leaves the image
30     blind_max_range: float = 1.8     # m, only if the tag was last seen at most this far away
31
32
33 @dataclass
34 class Observation:
35     now: float
36     visible: bool = False                # tag detected in the latest frame
37     bearing: float = 0.0                 # rad, valid when visible
38     distance: float = float('inf')       # m, ground-plane camera distance, valid when visible
39     tof: Optional[float] = None          # m, front ToF reading (None if unavailable)
40
41
42 def _clip(x, lo, hi):
```

```
    return max(lo, min(hi, x))


class NavStateMachine:
    def __init__(self, params: Params = None):
        self.p = params or Params()
        self.state = SEARCHING
        self.last_seen = None            # time of the last sighting
        self.last_dir = 1.0              # +1 CCW (tag was left), -1 CW
        self.centered_since = None
        self.traj = None
        self.traj_t0 = None
        self.arrival_reason = ''
        self.last_distance = float('inf')

    # -- helpers ---------------------------------------------------------------
    def _brake_hit(self, obs):
        if obs.visible and obs.distance <= self.p.brake_distance:
            return 'camera'
        if obs.tof is not None and obs.tof <= self.p.brake_distance:
            return 'tof'
        return ''

    def _enter_search(self):
        self.state = SEARCHING
        self.traj = None
        self.centered_since = None

    # -- main step -------------------------------------------------------------
    def update(self, obs: Observation):
        """Returns (v, w) to command this tick; self.state holds the current state."""
        p = self.p
        if self.state == ARRIVED:
            return 0.0, 0.0

        if obs.visible:
            self.last_seen = obs.now
            self.last_distance = obs.distance
            if abs(obs.bearing) > 1e-3:
                self.last_dir = 1.0 if obs.bearing > 0 else -1.0

        # active brake: engages immediately from any state
        reason = self._brake_hit(obs)
        if reason:
            self.state, self.arrival_reason, self.traj = ARRIVED, reason, None
            return 0.0, 0.0

        lost = self.last_seen is None or (obs.now - self.last_seen) > p.lost_timeout
        # A floor tag leaves the image when the robot is close (the camera cannot look straight down
        # at its own feet).  While chasing, keep following the planned trajectory without the tag:
        # it already ends at the 20 cm standoff.
        if (lost and p.blind_chase and self.state == CHASING and self.traj is not None
                and self.last_distance <= p.blind_max_range):
            return self._chase(obs)
        if lost:
            if self.state != SEARCHING:
                self._enter_search()
            return 0.0, p.search_rate * self.last_dir

        # the tag is (recently) visible
        if self.state == SEARCHING:
            self.state = ALIGNING
            self.centered_since = None

        if self.state == ALIGNING:
            if not obs.visible:
                return 0.0, 0.0           # between frames: hold still, do not guess
            if abs(obs.bearing) <= p.align_tol:
                if self.centered_since is None:
                    self.centered_since = obs.now
                if obs.now - self.centered_since >= p.settle_time:
                    self.traj = FlatTrajectory(obs.distance, v_max=p.v_max)
                    self.traj_t0 = obs.now
                    self.state = CHASING
                    return self._chase(obs)
                return 0.0, 0.0           # settling, in place
            self.centered_since = None
            w = _clip(p.align_gain * obs.bearing, -p.align_max_rate, p.align_max_rate)
            if abs(w) < p.align_min_rate:
                w = p.align_min_rate * (1.0 if w >= 0 else -1.0)
            return 0.0, w

        # CHASING
        if obs.visible and abs(obs.bearing) > p.realign_tol:
            self.state, self.traj, self.centered_since = ALIGNING, None, None
            return 0.0, 0.0
        return self._chase(obs)

    def _chase(self, obs):
        t = obs.now - self.traj_t0
        if self.traj.finished(t):
            self.state, self.traj = ARRIVED, None
            self.arrival_reason = 'trajectory end' if obs.visible else 'trajectory end, tag out of view'
            return 0.0, 0.0
        v, w_ff, _, _ = self.traj.sample(t)
        w = w_ff
        if obs.visible:
            w += self.p.chase_bearing_gain * obs.bearing
        return min(v, self.p.v_max), _clip(w, -self.p.chase_max_w, self.p.chase_max_w)
```

tag_detector.py

```
"""AprilTag detection (OpenCV aruco AprilTag dictionaries) and floor-plane geometry."""
import math
from typing import Optional

import cv2
import numpy as np

FAMILIES = {
    'tag16h5': 'DICT_APRILTAG_16h5', 'tag25h9': 'DICT_APRILTAG_25h9',
    'tag36h10': 'DICT_APRILTAG_36h10', 'tag36h11': 'DICT_APRILTAG_36h11',
}

# optical frame (x right, y down, z forward) -> base frame (x forward, y left, z up)
R_OPTICAL_TO_BASE = np.array([[0.0, 0.0, 1.0],
                              [-1.0, 0.0, 0.0],
                              [0.0, -1.0, 0.0]])


def camera_rotation(pitch_down):
    """Rotation optical -> base for a camera tilted down by `pitch_down` rad."""
    c, s = math.cos(pitch_down), math.sin(pitch_down)
    ry = np.array([[c, 0.0, s], [0.0, 1.0, 0.0], [-s, 0.0, c]])  # +y rotation tilts +x down
    return ry @ R_OPTICAL_TO_BASE


def ground_range_bearing(p_cam, r_base_cam):
    """Horizontal distance and bearing of a point given in the camera optical frame.

    r_base_cam rotates optical vectors into the base frame.  Bearing > 0 = target on the LEFT.
    Range is measured from the camera, projected onto the floor plane.
    """
    p = r_base_cam @ np.asarray(p_cam, dtype=float)
    return math.hypot(p[0], p[1]), math.atan2(p[1], p[0])


class TagDetection:
    def __init__(self, tag_id, corners, tvec, rvec):
        self.tag_id, self.corners, self.tvec, self.rvec = tag_id, corners, tvec, rvec


class TagDetector:
    def __init__(self, family='tag36h11', tag_id=3, tag_size=0.10):
        name = FAMILIES[family]
        self.tag_id, self.size = tag_id, tag_size
        self._dict = cv2.aruco.getPredefinedDictionary(getattr(cv2.aruco, name))
        # OpenCV 4.6 (Ubuntu 24.04 / ROS Jazzy) segfaults when a DetectorParameters() built with
        # the constructor is modified; its _create() factory is safe.  4.7+ only has the constructor.
        self._params = (cv2.aruco.DetectorParameters_create()
                        if hasattr(cv2.aruco, 'DetectorParameters_create')
                        else cv2.aruco.DetectorParameters())
        self._params.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_APRILTAG
        self._new_api = hasattr(cv2.aruco, 'ArucoDetector')
        if self._new_api:
            self._detector = cv2.aruco.ArucoDetector(self._dict, self._params)
        s = tag_size / 2.0
        self._obj = np.array([[-s, s, 0], [s, s, 0], [s, -s, 0], [-s, -s, 0]], dtype=np.float32)

    def detect(self, gray, camera_matrix, dist_coeffs=None, scales=(1.0, 2.0)) -> Optional[TagDetection]:
        """Pose of the configured tag id, or None if it is not in the image.

        A small tag (about 30 px per side or less, i.e. a 15 cm tag beyond ~1.3 m at 360p) is often missed
        at native size but found after enlarging the image, so each scale is tried in order and only the
        first success is used.  Corners are mapped back to native pixels before the pose is solved."""
        for scale in scales:
            img = gray if scale == 1.0 else cv2.resize(gray, None, fx=scale, fy=scale, interpolation=cv2.INTER_CUBIC)
            det = self._detect_once(img, camera_matrix, dist_coeffs, scale)
            if det is not None:
                return det
        return None

    def _detect_once(self, gray, camera_matrix, dist_coeffs, scale):
        if self._new_api:
            corners, ids, _ = self._detector.detectMarkers(gray)
        else:
            corners, ids, _ = cv2.aruco.detectMarkers(gray, self._dict, parameters=self._params)
        if ids is None:
            return None
        for c, i in zip(corners, ids.flatten()):
            if self.tag_id is not None and int(i) != self.tag_id:
                continue
            img = (c.reshape(4, 2) / scale).astype(np.float32)
            dist = np.zeros(5) if dist_coeffs is None else dist_coeffs
            ok, rvec, tvec = cv2.solvePnP(self._obj, img, camera_matrix, dist, flags=cv2.SOLVEPNP_IPPE_SQUARE)
            if ok:
                return TagDetection(int(i), img, tvec.reshape(3), rvec.reshape(3))
        return None
```

nav_node.py

```
"""ROS 2 node: AprilTag search, in-place alignment, flat-output pursuit, standoff brake,
and 1 Hz motor-voltage logging for the RoboMaster EP.

Subscribes : camera/image_color, camera/camera_info (sensor_msgs), range_0 (ToF, optional), TF
Publishes  : cmd_vel (geometry_msgs/Twist), target_arm_position (geometry_msgs/Point, at start)
"""
import csv
```

```
 8 import math
 9 import os
10 import threading
11 import time
12
13 import cv2
14 import numpy as np
15 import rclpy
16 from cv_bridge import CvBridge
17 from geometry_msgs.msg import Point, Twist
18 from rclpy.node import Node
19 from rclpy.qos import qos_profile_sensor_data
20 from sensor_msgs.msg import CameraInfo, Image, Range
21 import tf2_ros
22
23 from .state_machine import ARRIVED, NavStateMachine, Observation, Params
24 from .tag_detector import TagDetector, camera_rotation, ground_range_bearing
25 from .wheel_voltage import MotorVoltageModel, wheel_rates
26
27 CSV_HEADER = ['t_s', 'V1_V', 'V2_V', 'V3_V', 'V4_V', 'state',
28               'v_mps', 'w_radps', 'distance_m', 'bearing_rad', 'tof_m',
29               'w1_radps', 'w2_radps', 'w3_radps', 'w4_radps']
30
31
32 def quat_to_matrix(q):
33     x, y, z, w = q.x, q.y, q.z, q.w
34     return np.array([[1 - 2 * (y * y + z * z), 2 * (x * y - z * w), 2 * (x * z + y * w)],
35                      [2 * (x * y + z * w), 1 - 2 * (x * x + z * z), 2 * (y * z - x * w)],
36                      [2 * (x * z - y * w), 2 * (y * z + x * w), 1 - 2 * (x * x + y * y)]])
37
38
39 class GrabThread(threading.Thread):
40     """Optional direct OpenCV capture (camera_source:=cv2).  Implements the assignment hint:
41     a short sleep between cap.grab() calls lets the decoder recycle its buffers."""
42
43     def __init__(self, url):
44         super().__init__(daemon=True)
45         self.cap, self.frame, self.lock, self.running = cv2.VideoCapture(url), None, threading.Lock(), True
46
47     def run(self):
48         while self.running:
49             if self.cap.grab():
50                 ok, frame = self.cap.retrieve()
51                 if ok:
52                     with self.lock:
53                         self.frame = frame
54             time.sleep(0.010)
55
56     def latest(self):
57         with self.lock:
58             return self.frame
59
60
61 class FlatnessNav(Node):
62     def __init__(self):
63         super().__init__('flatness_nav')
64         d = self.declare_parameter
65         self.tag_family = d('tag_family', 'tag36h11').value
66         self.tag_id = d('tag_id', 3).value
67         self.tag_size = d('tag_size', 0.10).value          # metres, printed black-square side: MEASURE IT
68         self.image_topic = d('image_topic', 'camera/image_color').value
69         self.info_topic = d('info_topic', 'camera/camera_info').value
70         self.tof_topic = d('tof_topic', 'range_0').value
71         self.use_tof = d('use_tof', True).value
72         self.base_frame = d('base_frame', 'base_link').value
73         self.camera_frame = d('camera_frame', 'camera_optical_link').value
74         # The camera is fixed to the chassis (verified from images), so the arm/URDF TF is NOT used by default.
75         self.fallback_pitch_deg = d('camera_pitch_deg', 20.0).value   # measured: about 17-23 deg below horizontal
76         self.use_tf = d('use_tf_camera', False).value
77         self.control_rate = d('control_rate', 20.0).value
78         self.csv_path = d('csv_path', 'motor_voltage_log.csv').value
79         self.fast_path = d('fast_csv_path', 'motor_voltage_fast.csv').value   # 20 Hz log for the v(t), V1(t) plot
80         self.set_arm = d('set_arm_pose', True).value
81         self.arm_x, self.arm_z = d('arm_x_m', 0.105).value, d('arm_z_m', 0.030).value
82         self.camera_source = d('camera_source', 'ros').value          # 'ros' or 'cv2'
83         self.camera_url = d('camera_url', '').value
84         self.detect_fresh = d('detection_fresh_s', 0.20).value
85         fsm_params = Params()
86         for name in Params.__dataclass_fields__:
87             setattr(fsm_params, name, d(f'fsm.{name}', getattr(fsm_params, name)).value)
88         self.fsm = NavStateMachine(fsm_params)
89         self.voltage = MotorVoltageModel(d('kv', 0.38).value, d('ka', 0.06).value)
90         self.r = d('wheel_radius', 0.05).value
91         self.lc = d('chassis_radius', 0.20).value
92
93         self.detector = TagDetector(self.tag_family, self.tag_id, self.tag_size)
94         self.bridge = CvBridge()
95         self.K, self.dist = None, None
96         self.det_time, self.bearing, self.distance = -1e9, 0.0, float('inf')
97         self.tof = None
98         self.r_bc, self.tf_ok = camera_rotation(math.radians(self.fallback_pitch_deg)), False
99         self.tf_buffer = tf2_ros.Buffer()
100        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
101        self.grab = None
102
103        self.cmd_pub = self.create_publisher(Twist, 'cmd_vel', 10)
104        self.arm_pub = self.create_publisher(Point, 'target_arm_position', 1)
105        self.create_subscription(CameraInfo, self.info_topic, self.on_info, 1)
106        if self.camera_source == 'ros':
107            self.create_subscription(Image, self.image_topic, self.on_image, qos_profile_sensor_data)
108        else:
```

```
109             self.grab = GrabThread(self.camera_url)
110             self.grab.start()
111         if self.use_tof:
112             self.create_subscription(Range, self.tof_topic, self.on_range, qos_profile_sensor_data)
113 
114         self.t_start = self.now()
115         self.prev_tick = self.t_start
116         self.next_log = 0.0
117         self.volts, self.omegas = (0.0,) * 4, (0.0,) * 4
118         self.v_cmd = self.w_cmd = 0.0
119         self.stop_ticks = 0
120         self.finished = False
121         self._csv_file = open(self.csv_path, 'w', newline='')
122         self._csv = csv.writer(self._csv_file)
123         self._csv.writerow(CSV_HEADER)
124         self._fast_file = open(self.fast_path, 'w', newline='')
125         self._fast = csv.writer(self._fast_file)
126         self._fast.writerow(['t_s', 'v_mps', 'w_radps', 'V1_V', 'V2_V', 'V3_V', 'V4_V', 'state'])
127         self.create_timer(1.0 / self.control_rate, self.tick)
128         if self.set_arm:
129             self._arm_sends = 0
130             self.create_timer(1.0, self.send_arm)
131         self.get_logger().info(
132             f'Tag {self.tag_family} id={self.tag_id} size={self.tag_size} m. '
133             f'Table header: t(s) V1 V2 V3 V4 state. CSV -> {os.path.abspath(self.csv_path)}')
134         if self.tag_size == 0.10:
135             self.get_logger().warn('tag_size is the 0.10 m default: set it to the MEASURED tag side length.')
136 
137     # ---- helpers ------------------------------------------------------------------
138     def now(self):
139         return self.get_clock().now().nanoseconds * 1e-9
140 
141     def send_arm(self):
142         """Arm pose (X=105 mm, Y=30 mm): the driver's second coordinate is the vertical 'z'."""
143         self.arm_pub.publish(Point(x=self.arm_x, y=0.0, z=self.arm_z))
144         self._arm_sends += 1
145 
146     def on_info(self, msg):
147         self.K = np.array(msg.k, dtype=float).reshape(3, 3)
148         self.dist = np.array(msg.d, dtype=float) if len(msg.d) else None
149 
150     def on_range(self, msg):
151         self.tof = None if msg.range >= msg.max_range - 1e-3 else float(msg.range)
152 
153     def update_camera_rotation(self):
154         try:
155             tf = self.tf_buffer.lookup_transform(self.base_frame, self.camera_frame, rclpy.time.Time())
156             self.r_bc, self.tf_ok = quat_to_matrix(tf.transform.rotation), True
157         except Exception:
158             pass   # keep the fixed-pitch fallback
159 
160     def process_frame(self, frame):
161         if self.K is None:
162             return
163         gray = frame if frame.ndim == 2 else cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
164         det = self.detector.detect(gray, self.K, self.dist)
165         if det is None:
166             return
167         if self.use_tf:
168             self.update_camera_rotation()
169         self.distance, self.bearing = ground_range_bearing(det.tvec, self.r_bc)
170         self.det_time = self.now()
171 
172     def on_image(self, msg):
173         try:
174             self.process_frame(self.bridge.imgmsg_to_cv2(msg, 'bgr8'))
175         except Exception as exc:    # a bad frame must never stop the control loop
176             self.get_logger().warn(f'frame skipped: {exc}', throttle_duration_sec=2.0)
177 
178     # ---- control loop -------------------------------------------------------------
179     def tick(self):
180         if self.finished:
181             return
182         if self.grab is not None and self.grab.latest() is not None:
183             self.process_frame(self.grab.latest())
184         now = self.now()
185         dt = max(now - self.prev_tick, 1e-3)
186         self.prev_tick = now
187         visible = (now - self.det_time) <= self.detect_fresh
188         obs = Observation(now=now, visible=visible, bearing=self.bearing if visible else 0.0,
189                           distance=self.distance if visible else float('inf'), tof=self.tof)
190         v, w = self.fsm.update(obs)
191         self.v_cmd, self.w_cmd = v, w
192         tw = Twist()
193         tw.linear.x, tw.angular.z = float(v), float(w)
194         self.cmd_pub.publish(tw)
195         self.omegas = wheel_rates(v, w, self.r, self.lc)
196         self.volts = self.voltage.step(self.omegas, dt)
197 
198         t_rel = now - self.t_start
199         self._fast.writerow([f'{t_rel:.3f}', f'{v:.4f}', f'{w:.4f}', *[f'{x:.4f}' for x in self.volts], self.fsm.state])
200         if self.fsm.state == ARRIVED:
201             self.stop_ticks += 1
202             if self.stop_ticks == 3:      # let the derivative term settle: voltages are exactly 0
203                 self.log_row(t_rel, f'ARRIVED ({self.fsm.arrival_reason})', visible)
204                 self.get_logger().info(f'Halted at {t_rel:.1f} s ({self.fsm.arrival_reason}); '
205                                        f'log saved to {self.csv_path}')
206                 self.finished = True
207                 self._csv_file.close()
208                 self._fast_file.close()
209             return
```

```
        if t_rel >= self.next_log:
            self.log_row(math.floor(t_rel + 1e-6), self.fsm.state, visible)
            self.next_log += 1.0

    def log_row(self, t, state, visible):
        row = [f'{t:.1f}' if isinstance(t, float) else t, *[f'{x:.3f}' for x in self.volts], state,
               f'{self.v_cmd:.3f}', f'{self.w_cmd:.3f}',
               f'{self.distance:.3f}' if visible else '', f'{self.bearing:.3f}' if visible else '',
               f'{self.tof:.3f}' if self.tof is not None else '',
               *[f'{x:.3f}' for x in self.omegas]]
        self._csv.writerow(row)
        self._csv_file.flush()
        self.get_logger().info(f'{row[0]:>5} s | V1..V4 = ' + ' '.join(f'{x:7.3f}' for x in self.volts)
                               + f' V | {state}')

    def shutdown(self):
        """Always leave the robot stopped."""
        for _ in range(3):
            self.cmd_pub.publish(Twist())
        if self.grab is not None:
            self.grab.running = False
        for f in (self._csv_file, self._fast_file):
            if not f.closed:
                f.close()


def main(args=None):
    rclpy.init(args=args)
    node = FlatnessNav()
    try:
        # exit by itself shortly after the robot has halted, so the caller does not wait for a timeout
        while rclpy.ok() and not node.finished:
            rclpy.spin_once(node, timeout_sec=0.1)
        t_end = time.time() + 1.0
        while rclpy.ok() and time.time() < t_end:
            rclpy.spin_once(node, timeout_sec=0.1)     # let the final zero commands go out
    except KeyboardInterrupt:
        pass
    finally:
        node.shutdown()
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Build, test and run

```
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install && source install/setup.bash
python -m pytest -q src/rm_flatness_nav/test                              # 18 tests
ros2 launch robomaster_ros ep.launch conn_type:=rndis                     # or conn_type:=ap
ros2 launch rm_flatness_nav flatness_nav.launch.py tag_id:=3 tag_size:=0.15 \
    csv_path:=$PWD/motor_voltage_log.csv
```